

Chapter 22 - Music from BASIC and from each CPU

Part III has covered ten audio engines one at a time. This chapter pulls them together and shows how to drive them from the only two viewpoints that matter for a program: BASIC and machine language. The same engines are reachable from both, but the address shapes look very different.

22.1 The `SOUND PLAY` keyword

The fastest path to music from BASIC is the **media loader**: one keyword, one filename, and the loader picks the right engine. From the BASIC prompt:

```
SOUND PLAY "song.mod"  
SOUND PLAY "title.sid", 0  
SOUND PLAY "music.ahx", 1  
SOUND STOP
```

The loader dispatches on the file extension:

Extension	Engine	Chapter
.sid	SID family	15
.ym, .ay, .sndh, .vtx, .vt, .pt3, .pt2, .pt1, .stc, .sqt, .asc, .ftc, .vgm, .vgz, .snd	PSG / AY	13
.ted, .prg	TED audio	16
.ahx	AHX	18
.sap	POKEY	17
.mod	MOD	19
.wav	WAV	20

Subsong selection is the optional second argument to `SOUND PLAY`. Files without subsong support ignore it.

`SOUND STOP` stops every engine.

22.2 BASIC keyword summary by engine

Every engine that has dedicated BASIC verbs is listed below. The verbs are described in full in their own chapter.

Engine	Direct verbs	Chapter
SoundChip	SOUND, ENVELOPE, GATE	12
PSG / AY	PSG	13
SN76489	(none - use POKE to SN_PORT_WRITE)	14
SID family	SID	15
TED audio	TED TONE/TED VOL/TED NOISE/TED PLAY	16

Engine	Direct verbs	Chapter
POKEY + SAP	POKEY, SAP	17
AHX	AHX	18
MOD	(none - use POKE to the MOD_* registers)	19
WAV	(none - use POKE or SOUND PLAY)	20
Paula DMA	(none - use POKE to the Paula registers)	21

Mixer-wide effects (SOUND FILTER, SOUND REVERB, SOUND OVERDRIVE) are listed in Chapter 12.

22.3 Driving audio from machine language

In machine language the engine MMIO blocks are visible at the native addresses listed in their own chapters (0xF0xxx region). Writing them is the same as writing any other register: pick the address, write the value, and the chip reacts.

The relevant address bases for the IE64, IE32, and M68K CPUs (all of which see the full 32-bit address bus directly) are:

Engine	Address
AUDIO_CTRL	0xF0800
SoundChip channels	0xF0A80 (0–3), 0xF0C40 (4–6), 0xF0D40 (7–9)
SFX channels	0xF0E80
PSG	0xF0C00
SN76489	0xF0C30
POKEY	0xF0D00
SAP player	0xF0D10
SID	0xF0E00
SID2	0xF0E30
SID3	0xF0E50
TED audio	0xF0F00
AHX	0xF0B80
MOD	0xF0BC0
WAV	0xF0BD8
Paula DMA	0xF2260

The 6502 and Z80 cannot reach those addresses directly because their address spaces are too small. Each chip is also mapped into the smaller spaces, with these addresses or port numbers:

22.3.1 6502 audio map

The 6502 uses C64-style addresses for the heritage chips:

Engine	6502 address
PSG / SID	\$D400–\$D40F (PSG) and \$D500–\$D55F (SID, three instances at \$D500/\$D540/\$D580 on real C64; mapped contiguously here)
POKEY	\$D200–\$D209 (Atari convention)
TED audio	\$D600–\$D605 (mapped, not Plus/4-native)
VGA registers	\$D700–\$D70D

22.3.2 Z80 audio map

The Z80 uses explicit OUT (port), A / IN A, (port) instructions through a select/data pair per chip:

Engine	Select	Data
PSG / AY	\$F0	\$F1
TED audio	\$F2	\$F3
POKEY	\$D0	\$D1
SID	\$E0	\$E1

To write a register on a select/data engine: OUT (select), A selects which register; OUT (data), A writes the value.

The SN76489 has its own port pair and a different protocol. \$E4 is the data port: OUT (0xE4), A writes one command byte to the chip, and IN A, (0xE4) returns the last byte that was written (useful for read-back). \$E5 is the read-only status/ready port; bit 0 is set when the chip is ready to accept the next byte. There is no separate "select" port. A write to \$E4 either latches a new target register (when bit 7 of the byte is set) or appends data to the previously-latched register (when bit 7 is clear).

22.3.3 x86 audio map

The x86 coprocessor sees the same 0xFxxxx region as the IE64, IE32, and M68K, so it programs every chip at the addresses in the main table above.

22.4 A short cross-CPU example

A single program can be split across CPUs. One common arrangement:

- IE64 (the main CPU) sets up the picture and the menu.
- The Z80 coprocessor handles the PSG music in the background.
- The 6502 coprocessor plays a SID title theme on the menu.

Each CPU writes only its own engine and stays out of the others' register blocks. The shared mixer (Chapter 11) sums the results and produces a single output stream.

22.5 Picking an engine

The audio side has more choice than the video side. A few guidelines:

- For original music written for the IE: SoundChip (Chapter 12).

- For ZX Spectrum 128, Amstrad CPC, MSX, Atari ST chip-tunes: PSG (Chapter 13) or use `SOUND PLAY` on the `.ym/.ay/.vtx` file.
- For Sega Master System or BBC Micro chip-tunes: SN76489 (Chapter 14).
- For C64 chip-tunes: SID, SID2, SID3 (Chapter 15) or `SOUND PLAY` on the `.sid` file.
- For C16 / Plus-4 chip-tunes: TED audio (Chapter 16) or `SOUND PLAY` on the `.ted` file.
- For Atari 8-bit chip-tunes: POKEY (Chapter 17) or `SOUND PLAY` on the `.sap` file.
- For Amiga AHX music: AHX (Chapter 18) or `SOUND PLAY` on the `.ahx` file.
- For Amiga ProTracker music: MOD (Chapter 19) or `SOUND PLAY` on the `.mod` file.
- For sound effects and digitised speech: WAV (Chapter 20).
- For custom DMA-driven sample logic (your own tracker, your own streaming format): Paula DMA (Chapter 21).

Mix engines freely. A program can have a MOD playing in the background, an SFX channel for gunshots, a SoundChip channel for a tone, and a PSG playing arpeggios - all at once.

22.6 What comes next

Part IV opens with the memory model and the IE64 ISA. The audio side of Part III ends here.